

nanoGPT / Karpathy — train_gpt2.py

MASTER MEMORY-SHEET • code/term → one-line meaning • fast revision without reopening the source

Shape legend: B=batch, T=sequence length, C=model width, V=vocabulary size, nh=heads, hs=head size.

Core: tokens → embeddings → blocks → logits → loss → backward → accumulation → AdamW → validation/checkpoint → HellaSwag → generation → DDP.

1. Python / PyTorch essentials

Code / term	Meaning — remember this
<code>@dataclass</code>	Compact configuration class with an auto-generated constructor.
<code>nn.Module</code>	Base class for trainable PyTorch models; enables parameters, <code>.to()</code> , <code>.train()</code> , <code>.eval()</code> , <code>.state_dict()</code> .
<code>nn.Linear(a,b)</code>	Learned affine map from last dimension a to b.
<code>nn.Embedding(V,C)</code>	Integer ID → learned C-dimensional vector lookup.
<code>nn.ModuleDict</code> / <code>nn.ModuleList</code>	PyTorch-aware containers that register child modules.
<code>nn.LayerNorm(C)</code>	Normalize each token's feature vector; shape unchanged.
<code>nn.GELU()</code>	Nonlinearity that gives the MLP nonlinear expressive power.
<code>nn.Dropout(p)</code>	Randomly zeros activations in training; disabled in eval mode.
<code>assert condition</code>	Stop immediately if an assumption is false.
<code>view(...)</code>	Reshape tensor values without changing the values.
<code>transpose(a,b)</code>	Swap two tensor dimensions.
<code>contiguous()</code>	Make tensor memory layout contiguous so <code>view()</code> can safely follow.
<code>x[:, -n:]</code>	Keep all batches, only the last n sequence positions.
<code>size(-1)</code>	Size of the last tensor dimension.
<code>dim()</code>	Number of tensor dimensions.
<code>torch.no_grad()</code>	Inference without gradient-graph construction; saves memory/work.

2. Attention

Code / term	Meaning — remember this
<code>qkv = c_attn(x)</code>	One Linear layer creates Q, K, V together: $C \rightarrow 3C$.
<code>q,k,v = split(...)</code>	Separate the combined QKV features.
<code>C // nh</code>	Head size hs; model width divided evenly among heads.
<code>view(B,T,nh,hs).transpose(1,2)</code>	Split into heads and arrange as <code>(B,nh,T,hs)</code> .
<code>scaled_dot_product_attention(..., is_causal=True)</code>	Score → scale → block future tokens → softmax → weighted V.
$Q / K / V$	Q = what I seek; K = what each token advertises; V = information to retrieve.
Causal mask	Future-token scores are suppressed so a token cannot see future tokens.
Attention output	Weighted sum of Value vectors: the actual cross-token information gathering.
<code>transpose(...).contiguous().view(B,T,C)</code>	Merge all heads back to the model width.
<code>c_proj</code>	Mix information from attention heads; shape stays <code>(B,T,C)</code> .

3. MLP + Transformer Block

Code / term	Meaning — remember this
<code>c_fc = Linear(C,4C)</code>	Expand each token into a wider feature space.
<code>GELU</code>	Apply nonlinearity.
<code>c_proj = Linear(4C,C)</code>	Compress back to model width.
MLP	Processes each token independently; it does not mix tokens.
<code>x = x + attn(ln_1(x))</code>	Normalize → attention → add learned change to old x.
<code>x = x + mlp(ln_2(x))</code>	Normalize → per-token MLP → add learned change.
Residual	<code>x + change</code> preserves old information and helps gradient flow.
Block shape	Remains <code>(B,T,C)</code> ; blocks change meaning, not dimensions.

4. GPT assembly + forward

Code / term	Meaning — remember this
<code>`wte`</code>	Token embedding table: answers “what token?”.
<code>`wpe`</code>	Position embedding table: answers “where?”.
<code>`pos = arange(0,T,...)`</code>	Create position IDs 0...T-1 on the input device.
<code>`tok_emb = wte(idx)`</code>	<code>`(B,T)`</code> token IDs → <code>`(B,T,C)`</code> vectors.
<code>`pos_emb = wpe(pos)`</code>	<code>`(T)`</code> positions → <code>`(T,C)`</code> vectors.
<code>`x = tok_emb + pos_emb`</code>	Combine token identity + position via broadcasting.
<code>`for block in h`</code>	Run the representation through every Transformer block.
<code>`ln_f(x)`</code>	Final LayerNorm before prediction.
<code>`lm_head = Linear(C,V,bias=False)`</code>	Hidden vector → one raw score for every vocabulary token.
<code>`logits = lm_head(x)`</code>	Raw vocabulary scores; shape <code>`(B,T,V)`</code> , not probabilities.
Weight tying	Embedding matrix and output classifier share weights.
<code>`forward(idx, targets=None)`</code>	Targets present = training/loss; targets absent = inference/logits only.
<code>`assert T <= block_size`</code>	Reject sequences longer than the context window.

5. Logits → loss / shapes

Code / term	Meaning — remember this
<code>`logits.view(-1, logits.size(-1))`</code>	<code>`(B,T,V) → (B*T,V)`</code> ; flatten positions, preserve vocabulary.
<code>`targets.view(-1)`</code>	<code>`(B,T) → (B*T,)`</code> ; one target ID per prediction.
<code>`-1`</code> in <code>`view`</code>	Tell PyTorch to infer that dimension automatically.
<code>`size(-1)`</code>	Read the final dimension, here V.
<code>`cross_entropy`</code>	Compare each row of logits with its correct token ID and return scalar loss.
Logits intuition	Final linear layer is a learned weighted combination producing one score per vocabulary token.
<code>`logits[:, -1, :]'`</code>	Keep only the final position when predicting the next token.

6. AdamW / optimization

Code / term	Meaning — remember this
<code>`named_parameters()`</code>	Return every parameter with its name.
<code>`requires_grad`</code>	Keep only parameters that should learn.
<code>`p.dim() >= 2`</code>	Matrix-like weights → weight decay.
<code>`p.dim() < 2`</code>	Biases/LayerNorm-like 1D parameters → no weight decay.
Optimizer groups	Different parameter sets can receive different hyperparameters.
AdamW	Updates parameters using Adam gradient moments plus decoupled weight decay.
<code>`betas=(0.9,0.95)`</code>	Controls Adam's memory of past gradient statistics.
<code>`eps=1e-8`</code>	Numerical safety constant.
<code>`fused`</code>	Use optimized fused AdamW when supported on CUDA.
<code>`zero_grad()`</code>	Clear old gradients before the next update.
<code>`loss.backward()`</code>	Compute gradients; does not change weights.
<code>`optimizer.step()`</code>	Actually update the weights.

7. DataLoaderLite

Code / term	Meaning — remember this
<code>`load_tokens()`</code>	<code>`numpy`</code> shard → NumPy → int32 → PyTorch <code>`long`</code> tensor.
<code>`current_position`</code>	Where the next batch begins in the current shard.
<code>`buf = tokens[pos:pos+B*T+1]`</code>	Read one extra token so inputs and shifted targets can be formed.
<code>`x = buf[:-1].view(B,T)`</code>	Input sequence.
<code>`y = buf[1:].view(B,T)`</code>	Next-token targets; y is x shifted by one.
<code>`current_position += B*T*num_processes`</code>	Advance so DDP processes consume separate portions.
<code>`current_shard`</code>	Index of the currently loaded dataset chunk.
<code>`(shard+1) % len(shards)`</code>	Move to next shard and wrap around at the end.
Shard	One chunk/file of the huge tokenized dataset.

8. Gradient accumulation + DDP

Code / term	Meaning — remember this
RANK	Global process/GPU index.
LOCAL_RANK	GPU index on the current machine.
WORLD_SIZE	Total number of DDP processes.
<code>`master_process = rank == 0`</code>	Only rank 0 logs/checkpoints to avoid duplicates.
<code>`DDP(model, ...)`</code>	Each GPU has a model replica; DDP synchronizes gradients.
<code>`model.module`</code>	Original model inside the DDP wrapper.
<code>`total_batch_size`</code>	Desired effective tokens per optimizer update.
<code>`B*T`</code>	Tokens in one micro-batch per process.
<code>`grad_accum_steps`</code>	How many micro-batches combine into one effective batch.
<code>`loss /= grad_accum_steps`</code>	Accumulation adds gradients; division makes the result correspond to a mean.
<code>`require_backward_grad_sync`</code>	With accumulation, sync DDP gradients only on the final micro-step.
<code>`all_reduce(..., AVG)`</code>	Combine values across processes, e.g. average loss.
<code>`destroy_process_group()`</code>	Cleanly shut down DDP communication.

9. Training loop + schedule

Code / term	Meaning — remember this
<code>`model.train()`</code>	Training behavior; e.g. Dropout is active.
<code>`model.eval()`</code>	Evaluation behavior; e.g. Dropout is disabled.
<code>`autocast(..., bfloat16)`</code>	Use lower precision where supported for speed/memory efficiency.
<code>`clip_grad_norm(...,1.0)`</code>	Limit gradient norm to 1.0 to reduce unstable/exploding updates.
<code>`get_lr(step)`</code>	Compute the learning rate for the current step.
Warmup	Increase LR linearly from small to max LR at the start.
Cosine decay	After warmup, smoothly decrease LR toward min LR.
<code>`param_group['lr'] = lr`</code>	Apply the current LR to optimizer groups.
<code>`cuda.synchronize()`</code>	Wait for GPU work to finish before timing.
<code>`tokens_processed = B*T*accum*world_size`</code>	Total tokens processed in one optimizer step.
<code>`tokens_per_sec = tokens_processed/dt`</code>	Training throughput.

10. Validation + checkpoints

Code / term	Meaning — remember this
<code>`step % 250 == 0 or last_step`</code>	Run validation periodically and at the final step.
<code>`val_loader.reset()`</code>	Restart validation traversal from a known position.
<code>`no_grad()`</code>	Validation needs no gradients.
<code>`loss /= val_loss_steps`</code>	Average over validation batches.
<code>`loss.detach()`</code>	Use the tensor value without gradient history.
<code>`loss.item()`</code>	Extract a Python scalar for logging/storage.
Checkpoint	Stores model state/config plus step and validation loss.
<code>`torch.save(...)`</code>	Write checkpoint to disk.
Loss rule	Lower validation loss is better.

11. HellaSwag

Code / term	Meaning — remember this
Four candidates	Score each possible completion under the same context.
Shift logits/tokens	Align each prediction with the token it should predict.
<code>`reduction='none'`</code>	Keep per-token losses instead of immediately averaging.
Mask	Only completion tokens contribute to the candidate score.
<code>`sum_loss / mask.sum()`</code>	Average loss over the completion region.

Code / term	Meaning — remember this
<code>`argmin()`</code>	Choose the candidate with the lowest average loss.
Accuracy	Compare predicted candidate index with the correct label.

12. Generation / sampling

Code / term	Meaning — remember this
<code>`idx_cond = idx[:, -block_size:]`</code>	If context is too long, keep only the most recent context window.
<code>`logits[:, -1, :]'`</code>	Use the last position because it predicts the next token.
Temperature	<1 = sharper/more deterministic; >1 = flatter/more random.
<code>`softmax(logits, dim=-1)`</code>	Turn vocabulary logits into probabilities.
<code>`topk(probs,50)`</code>	Keep only the 50 most likely candidates.
<code>`multinomial(topk_probs,1)`</code>	Sample a candidate according to probability; not necessarily the maximum.
<code>`gather(topk_indices,-1,ix)`</code>	Convert sampled top-k position into the actual vocabulary token ID.
<code>`cat((xgen,xcol),dim=1)`</code>	Append the new token to the sequence.
Autoregressive	Predict → append → feed back → predict again.

13. Pretrained GPT-2

Code / term	Meaning — remember this
<code>`GPT.from_pretrained('gpt2')`</code>	Create local architecture and import learned GPT-2 weights.
GPT-2 config	gpt2 = 12 layers, 12 heads, 768 width; larger variants scale these.
<code>`model_hf.state_dict()`</code>	Get Hugging Face parameter dictionary.
Ignore mask buffers	Attention masks are buffers, not learned weights.
Transposed weight list	Conv1D-style GPT-2 matrices need orientation correction for vanilla Linear.
<code>`.t()`</code>	Transpose a 2-D weight matrix.
<code>`copy_(...)`</code>	Copy pretrained numerical values into local parameters.

14. Final mental model

Code / term	Meaning — remember this
Training	clear grads → batch → forward → logits → loss → backward → accumulate → clip → LR → optimizer.step.
Validation	eval → no_grad → batches → average loss → DDP reduce → log.
Checkpoint	save model/config/step/validation result.
Generation	context → last logits → temperature → softmax → top-k → sample → gather → append.
DDP	one model replica per GPU + different data slices + synchronized gradients.
Shape chain	$(B,T) \rightarrow (B,T,C) \rightarrow (B,T,V) \rightarrow (B^*T,V)$ vs targets $(B^*T,.)$.
Golden rules	Lower loss is better; backward computes gradients; optimizer.step changes weights; eval changes behavior; no_grad disables gradient tracking.

Ultra-fast final revision

- Attention = tokens communicate; MLP = each token transforms independently.
- Q = what I seek; K = what is advertised; V = what I retrieve.
- Embedding = ID → vector; logits = hidden vector → raw vocabulary scores.
- Softmax = logits → probabilities; cross-entropy = probability scores vs correct IDs → loss.
- Backward = gradients; optimizer.step = weight update.
- Gradient accumulation = many micro-batches → one effective large batch.
- DDP = model replica per GPU + synchronized gradients.
- Checkpoint = model/config + training position + validation result.
- HellaSwag = candidate with lowest completion loss wins.
- Generation = last logits → temperature → softmax → top-k → sample → gather → append.